Step 3 — Deciding What's Worth Remembering, and Writing It Back — Solution

> Back to the step · Hint 1 · Hint 2

Story: this adds a `write_back_node` that decides, for the user's most recent message, whether it actually holds a durable fact worth remembering — and if so, saves it under a stable, reusable key, so a later correction updates the same entry instead of creating a duplicate. The real skill this step teaches is saying *no* far more often than yes. **If not:** the Store would fill up with "pasta tonight" noise and three contradictory spellings of the user's own name, none of them cleaned up.

`src/memory/writer.py` (new)

Story: this is a genuinely structured decision, not a guess. `decide_memory` asks the model to return a typed `MemoryDecision` — whether to save at all, under which stable key, and what fact to actually store. The prompt does the hard judgment work: durable facts yes, passing ephemera no, and reuse a consistent key like `preferred_name` so a later change updates it rather than adding a new entry. **If not:** parsing free text for "should I remember this?" would be fragile, and nothing would enforce the stable keys that make updates work correctly.

```
# src/memory/writer.py
# Decide whether the user's last message holds a durable fact worth keeping,
# and under which stable key. The model returns a checked MemoryDecision.
from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel

from src.utils.prompts import load_prompt


class MemoryDecision(BaseModel):
    should_remember: bool
    key: str
    fact: str


def decide_memory(model, message):
    prompt = ChatPromptTemplate.from_template(load_prompt("writer"))
```

```
    chain = prompt | model.with_structured_output(MemoryDecision)
    return chain.invoke({"message": message})
```

```
# src/prompts/writer.txt
Decide whether the user's message contains a DURABLE fact worth remembering
long-term about this user.

Remember: a name, a lasting preference, an allergy, a standing choice.
Do NOT remember: today's plans, one-off questions, small talk, anything
temporary.

If it is worth remembering, set should_remember true, choose a short STABLE
key (reuse "preferred_name" for the user's name so a change updates it), and
write the fact as a short sentence ("Name is Sam"). Otherwise set
should_remember false with empty key and fact.

User message: {message}
```

`src/nodes.py` — add `write_back_node`

Story: after the chat response, the node looks at the user's last message, asks `decide_memory` to judge it, and saves only if the decision said yes — under the exact key the decision chose, so a later value for that same key cleanly replaces the old one. **If not:** the assistant would either never learn anything about the user on its own, or would hoard absolutely everything said to it.

```
# src/nodes.py  (add this; recall_node and chat_node unchanged)
from langchain_core.messages import HumanMessage

from src.memory.store import save_fact
from src.memory.writer import decide_memory


def make_write_back_node(model):
    def write_back_node(state, *, store):
        last_user = None
        for message in reversed(state["messages"]):
            if isinstance(message, HumanMessage):
                last_user = message
                break
        if last_user is None:
            return {}
        decision = decide_memory(model, last_user.content)
        if decision.should_remember:
            save_fact(store, state["user_id"], decision.key, decision.fact)
```

```
        return {}
    return write_back_node
```

(The node needs both the model and the store: the model is captured by the closure, and the store is injected because the inner function declares a `store` parameter.)

`src/graph.py` — chat then write back

```
# src/graph.py  (Step 3 version)
from langgraph.graph import END, START, StateGraph

from src.nodes import make_chat_node, make_write_back_node, recall_node
from src.state import ChatState


def build_graph(model, store, checkpointer):
    graph = StateGraph(ChatState)
    graph.add_node("recall", recall_node)
    graph.add_node("chat", make_chat_node(model))
    graph.add_node("write_back", make_write_back_node(model))
    graph.add_edge(START, "recall")
    graph.add_edge("recall", "chat")
    graph.add_edge("chat", "write_back")
    graph.add_edge("write_back", END)
    return graph.compile(checkpointer=checkpointer, store=store)
```

Proving it (part of the integration test)

The behaviour that matters needs the real model, so it's checked in `tests/integration/test_live_memory.py` (full file in Step 4's solution):

- "my name is Sam" → a `preferred_name` fact is saved; a brand-new thread recalls it.
- "I think I'll have pasta tonight" → **nothing** is saved (ephemeral).
- "actually, call me Sammy" → the `preferred_name` fact is **updated**, not duplicated (one fact, now "Sammy").

Expected behaviour, with `decide_memory` driving the write-back:

```
You: my name is Sam            -> saves preferred_name = "Name is Sam"
You: I'll have pasta tonight   -> saves nothing
You: actually call me Sammy    -> updates preferred_name = "Name is Sammy"
(new thread) what's my name?   -> the model answers "Sammy"
```

The whole point is the ratio: most messages save nothing. A `write_back` that saves on every turn isn't memory, it's a junk drawer — the `decide_memory` prompt's job is to say no.